

Техническое задание «Боги» — Paper 26.2

Плагин механики персональной реальности и явлений

Версия ТЗ: 1.0 | Целевая платформа: Paper 26.2

1. Цель проекта

- Реализовать для Paper 26.2 серверную систему «Боги», в которой игровой мир может по-разному выглядеть для разных игроков.
- Основная идея: событие, NPC, моб, эффект или объект может существовать только для выбранного игрока либо группы игроков. Остальные игроки не должны получать информацию об этом объекте и не должны видеть его стандартными средствами клиента.
- Система должна использоваться для сюжетных явлений, галлюцинаций, персональных сцен, божественных вмешательств и других механик, где один игрок наблюдает то, чего не наблюдают остальные.

2. Основные механики

- 2.1. Персональное исчезновение: выбранная сущность или объект становится невидимым для заданных игроков, при этом для остальных игроков продолжает существовать.
- 2.2. Персональное появление: выбранная сущность или объект отображается только заданным игрокам.
- 2.3. Фантомные mobs: создание клиентски-видимых мобов, которые существуют только для выбранных игроков и используются для галлюцинаций/видений.
- 2.4. Персональные эффекты: возможность показывать выбранному игроку частицы, звуки, сообщения, анимации и другие визуальные/аудиальные признаки события.
- 2.5. Управление видимостью должно работать независимо для каждого игрока: если А видит объект, это не означает, что В обязан его видеть.

3. Требования к персональной видимости

- Для каждой управляемой сущности должна поддерживаться матрица видимости «сущность → игрок».
- Должны поддерживаться режимы: VISIBLE — видят все; HIDDEN — не видит никто из управляемой группы; ONLY — видит только указанный список игроков; EXCEPT — видят все, кроме указанного списка.
- При смене режима игроки должны получать корректное обновление состояния без необходимости перезаходить на сервер.
- Система не должна удалять сущность из мира, если требуется только изменить её видимость для конкретного игрока.
- После смерти, телепортации, смены мира, перезагрузки чанка или повторного входа игрока состояние видимости должно восстанавливаться автоматически.

4. Фантомные mobs

- Фантомный моб — объект, предназначенный для отображения только одному игроку или группе игроков.
- По возможности фантомный моб не должен становиться полноценной серверной сущностью, если для сценария достаточно клиентской иллюзии.
- Фантом должен иметь: уникальный ID, тип моба, позицию, поворот, имя/отображаемое имя, экипировку при необходимости, состояние AI/анимации и список наблюдателей.
- Поддерживать создание, перемещение, изменение состояния, телепортацию и удаление фантома.
- При потере наблюдателя фантом не должен продолжать отправлять ему пакеты.
- При повторном подключении игрока персональные фантомы должны восстанавливаться согласно активному сценарию.

5. Сценарная система

- Механики должны быть доступны из единой API/сервисной прослойки, чтобы другие плагины могли запускать явления без прямой работы с пакетами.
- Минимальные операции API: `createPhantom()`, `removePhantom()`, `movePhantom()`, `showTo()`, `hideFrom()`, `showOnlyTo()`, `hideFromAll()`, `playPersonalEffect()`.
- Каждое событие должно иметь идентификатор, владельца/источник, список наблюдателей и время жизни.
- Должна поддерживаться автоматическая очистка временных объектов по TTL.
- Сценарий не должен зависеть от конкретного игрока, если это не требуется логикой события.

6. Команды администратора

- `/god phantom create` — создать фантом.
- `/god phantom remove` — удалить фантом.
- `/god phantom move` — переместить фантом.
- `/god phantom show` — показать фантом игроку.
- `/god phantom hide` — скрыть фантом от игрока.
- `/god visibility only` — оставить отображение только указанному игроку.
- `/god visibility except` — скрыть от указанного игрока.
- `/god debug` — включить/выключить диагностический режим.
- `/god reload` — перечитать конфигурацию без перезапуска сервера.

7. Права

- `god.admin` — полный доступ к системе.
- `god.phantom` — создание и управление фантомами.
- `god.visibility` — управление персональной видимостью.
- `god.debug` — доступ к диагностике.
- `god.reload` — перезагрузка конфигурации.

8. Конфигурация

- Конфигурация должна быть вынесена в YAML.
- Обязательные параметры: максимальное количество фантомов на игрока, глобальный лимит фантомов, TTL, радиус загрузки, режим логирования, автоматическая очистка, поведение после рестарта.
- Все лимиты должны проверяться до создания объекта.
- Ошибки конфигурации не должны приводить к падению сервера: использовать значения по умолчанию и выводить понятное сообщение в консоль.

9. Производительность

- Основная логика не должна выполнять тяжёлые операции в главном тике чаще, чем необходимо.
- Не допускать циклического перебора всех игроков для каждого фантома на каждом тике. Использовать списки наблюдателей, кэширование и обновление только при изменении состояния.
- Не отправлять повторно одинаковое состояние игроку.
- Все временные объекты должны корректно освобождать ресурсы.
- Плагин должен нормально работать минимум на сервере с 100+ игроками при разумном количестве активных персональных объектов.

10. Безопасность и устойчивость

- Проверять существование игрока, мира, сущности и корректность всех входных параметров.
- Не доверять данным из команд и внешнего API.
- Не допускать утечек персональных объектов другим игрокам после смены наблюдателя.
- При отключении плагина все созданные временные ресурсы должны корректно очищаться.
- Ошибки отдельного сценария не должны ломать обработку остальных сценариев.

11. Совместимость

- Целевая платформа: Paper 26.2.
- Не использовать NMS напрямую без изоляции в отдельном compatibility-слое.
- Пакетную/протокольную часть реализовать через стабильный abstraction layer; все версии-специфичные участки вынести в отдельные классы.
- API плагина должен быть минимально связан с внутренностями Paper, чтобы последующее обновление версии не требовало переписывать всю систему.

12. Логирование и диагностика

- Логировать создание и удаление фантомов, изменение наблюдателей и критические ошибки.
- Debug-режим должен позволять определить: ID объекта, тип, владельца, координаты, число наблюдателей и причину удаления.
- Предусмотреть команду/метод получения списка активных персональных объектов.

13. Критерии приёмки

- Игрок А может видеть фантомного моба, которого игрок В не видит.

- Игрок В не получает обычное серверное отображение персонального объекта А.
- После телепортации, смены мира, смерти и повторного входа состояние видимости восстанавливается.
- Фантом можно создать, переместить и удалить без перезапуска сервера.
- Фантомы не создают лишних полноценных серверных сущностей без необходимости.
- При 100+ игроках и большом количестве персональных объектов нет заметного постоянного роста нагрузки из-за полного перебора игроков каждый тик.
- После /reload или перезагрузки конфигурации плагин не теряет активные состояния, если это предусмотрено конфигурацией.
- При отключении плагина временные объекты и слушатели корректно очищаются.

14. Этапы разработки

- Этап 1: базовый plugin core, конфигурация, команды и permission.
- Этап 2: менеджер персональной видимости.
- Этап 3: система фантомных мобов и packet abstraction.
- Этап 4: персональные эффекты и сценарный API.
- Этап 5: восстановление состояния и обработка жизненного цикла игрока.
- Этап 6: оптимизация, диагностика и нагрузочное тестирование.
- Этап 7: документация API и финальная сборка под Paper 26.2.